

赛道三 自动数据标注

技术报告 (团队: 玉兔)

一、问题背景

预训练的语言模型, 虽然构建了一个很强大的语言符号处理系统, 对于不同的输入计算的中间变量数据是各不相同的, 但是模型参数是固定的, 推理过程是可预测的、可观察的、可重复的; 虽然可以控制 **temperature**、**topk** 等手段, 对同一个输入产生有差异的结果, 但是在实际上, 这是对模型最终输出结果的人为干预, 随机改变了模型结果的表达而已, 也必定与期望有较大的差距, 因此, 天然的存在稳定性差和准确率低的问题。

对于以上问题, 如何利用技巧和模拟, 通过多步骤操作模型自动标注, 提高模型输出的稳定性与准确率。

二、解决方案

1) 核心理念

虽然预训练语言模型的推理过程和处理机制, 可计算几乎所有的可能性和注意力, 推理过程是局限的、固定的、被动的、单向的, 但是经过大量语言符号数据的反馈学习训练, 依然能在一定程度上揭示了自然语言的递归结构与自相关性, 因此大模型在阅读、联想、推理、推导、归纳等方面具备了一些本能能力。

自然语言本身具有严格的关联关系、逻辑规则和关键操作, 并且其关键操作最能准确反映我们经验世界, 而且很多操作方法都是隐性的, 并且对于同类特定问题的关键操作是相同的, 并且在人类的语言使用习惯中必须依赖大量的关键操作, 并且这种关键操作与实时环境有关, 甚至会与概率有关; 但是大语言模型推理过程都是局限的、固定的、被动的、单向的, 很显然无法稳定准确地触发对应的关键操作。如果能有办法让大模型准确触发对应的关键操作, 也许就能让大模型在符号世界与经验世界之间建立一个双向通信的有意义的信道, 从而与人类实现无障碍对话交流。

虽然大模型推理过程无法稳定准确地触发对应的关键操作, 如果我们能先提取出那些可以触发关键操作的提示词, 并且让其可增长可复用, 让新输入信息都附加这些提示词, 与过去的经历经验产生联系, 显示地触及较完整的关键操作提示词, 那么就赋予了大模型对过往经历的学习记忆能力, 也就能让大模型更稳定更准确地触发对应的关键操作, 也就能让大语言模型在理解、联想、推理、归纳等方面变得更稳定、更准确、更快速。

因为大模型推理过程是固定的、被动的、单向的, 也就是可预测的、可观察的、可重复的过程, 因而可以利用这个大模型特点, 通过不断推测和调整提示信息, 尝试让大模型提取出特定问题的关键操作的提示词, 比如, 通过大量 ICL 示例数据反复多次提取得到较全面的关键操作的提示词, 并且不断的提取和积累这些隐性的关键操作的提示词, 并且应用到新输入信息的推理中, 从而建立当前推理与过往推理之间的经验迁移能力, 将这些隐性的关键操作贯穿到所有的推理过程, 也就能让大模型更稳定更准确触发这些隐性的关键操作, 也就能提高模型推理的稳定性和准确率。

2) 实施方案

因为预训练大语言模型经过大量语言符号数据反馈学习训练, 所以大语言模型必然对语义语法极度敏感, 混淆和歧义很容易让大语言模型推理出错, 因此, 必须设计好提示词的关联关系、逻辑规则和目标要求, 以给大模型传递更准确的信息, 从而提高模型推理的稳定性和准确率。其中, 关联关系有很大的局部性和差异性, 逻辑规则是人们可以掌控的, 隐性的关键操作在同类问题上相对一致的, 因此可以在第一步提取出能触发隐性的关键操作的提示词, 在第二步应用这些提示词到新的输入信息推理中, 从而等效于主动触发这些隐性的关键操作, 提高模型的稳定性和准确率。

首先, 分析和设计模型处理的抽象思维过程, 确定隐性的关键操作。设计实验步骤和操作流程, 便于理论与实验进行对照与验证, 以及进行理论与实验的迭代与优化。

然后, 对辅助提示词设计和 ICL 示例数据处理很关键。辅助提示必须用严谨的语义语法将关联关系、逻辑规则 (例如条件限制)、目标要求等表述清晰, 遣词造句尽量避免混淆与歧义, 将我们的信息传准确的递给大模型, **准确周密的提示词既能让结果更准确与稳定, 也能让推理更快速**; 将 ICL 示例数据分割为许多个小批次, 并且采用不同的样本组合进行多轮提取, 既能让每个批次既满足上下文长度要求, 又能充分利用全部样本数据, 通过多次学习积累, 得到更全面的 key 操作提示词, 既能增量隐性的 key 操作提示词, 也能更具有实用性。

最后, 对于存在的多次或深层循环迭代的模型细节过程, 都用抽象的流程概念概括, 因为这些细节过程不是本次研究的重点, 本次主要研究核心环节与关键因素, 找到可干预可调控的关键操作, 以影响推理过程中那些细节过程的中间输出, 从而提高大模型推理最终输出的稳定性和准确率。由于大语言模型返回的数据格式不会总是符合要求的, 还需增加简单提取数据结果的逻辑与规则, 以及对于异常数据重新生成, 从而提高系统框架的稳定和准确率。

3) 操作准则

为了保证实施过程与设计方案一致, 以及对设计方案与表现效果进行质量检验和优化, 须遵循以下准则:

- a、需要代码简洁, 突出核心, 流程完整, 逻辑清晰, 参数友好, 便于快速开发和优化
- b、需要日志跟踪, 便于状态监控和查阅分析
- c、需要命令脚本, 便于部署和操作
- d、需要环境变量, 便于调试和环境迁移
- e、需要提示词的质量标准, 须输出准确稳定、推理快速、输出 **think** 内容言简意赅, 并且逻辑合理准确, 输出结果可重现, 甚至 **think** 内容可重现。
- f、需要输出结果的质量标准, 须与预期一致或十分接近, 以及与 **think** 内容的推理相符
- g、其他待补充

通过遵循以上准则, 对提示词和代码进行优化迭代。并且在优化提示词时, 通过实时打印模型输出内容, 观察其思考内容和输出结果, 可以理解分析模型推理的思考过程, 发现其中的关键控制因素, 以及推测提示词的影响方向和作用效果, 从而立即做出反应, 对提示词做出迅速微调和立即验证。

三、分析设计

首先，我们假设模型处理的抽象思维过程如下：

For _ IN RANGE(layer_size):



其中，

X 是特定问题的输入，
Definition 是人为设定的文本信息，
Association 是模型自发建立的关联关系并且与每个 x 输入**强相关**，与 Definition **弱相关**，
Operation 是模型自发采用的关键操作但是与每个 x 输入**弱相关**，与 Definition **强相关**，
Key+Value 是模型自发生成的关键信息并且与每个 x 输入**强相关**，与 Definition **强相关**，
Y 是特定问题的输出

然后，设计实验步骤和流程。因为 x 输入是任意的外界变量，Definition 是人为确定的，Association 和 Key+Val 是模型内变量，并且具有较大差异的。Operation 是模型内变量，与 Definition 高度相关的，并且相对统一和敏感，更容易人为干预和调节，因此更容易让大语言模型通过学习记忆找到特定问题的有效 Operation，并且与新的 X 输入信息一起提交给模型思考推理，从而提高模型推理的稳定性和准确率，还可以让用户通过查阅 Operation 来理解模型推理该领域问题的关键点，从而找到对提示词的优化方向和验证依据。本次实验用简单的文件保存与加载来模拟对 Operation 的记忆过程。

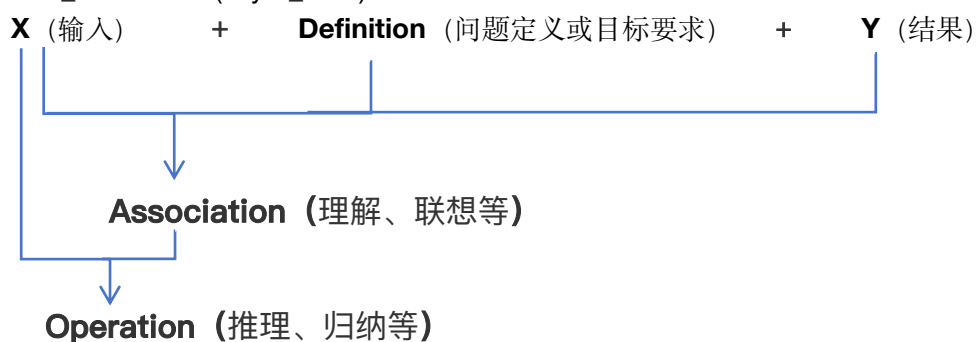
最后，实验步骤和流程如下：

第一步：基于 FlagScale + Qwen3-4B + NPU 部署大模型服务

第二步：利用每个特定问题的 ICL 长文案获取每个特定问题的关键操作 Operation

FOR (x, y) IN icl_examples:

FOR _ IN RANGE(layer_size):



其中，在第二步中利用 ICL 示例提取 Operation，并去重合并持久保存，其伪代码如下：

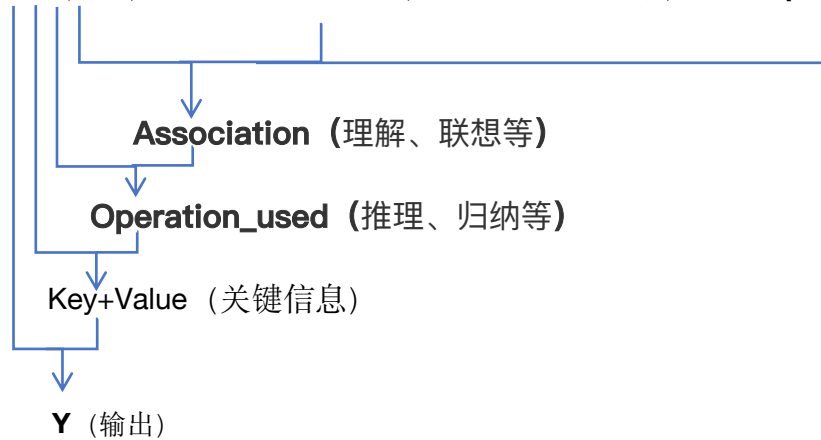
```
# 只展示核心的主要代码，表达主要逻辑规则
# 提取轮次（若 NPU 卡较少，为了加速设置为 N=1，若 NPU 卡较多 可以尝试多跑几遍）
N = 1
# 批次步长(例如 100、50、10 等)
batch_size = TASK_BATCH_SIZES[task_id]
task_operations = set()
for i in range(N):
    # 在重复采样时，打乱 ICL 的顺序，每轮即可得到不同样本组合的批次，让样本表现为多样性
    if i > 0:
        random.shuffle(icl_samples)
    for pos in range(0, icl_length, batch_size):
        # 第 (pos // batch_size + 1) 次提取 Operation
        icl_batch_data = select_examples(icl_examples[pos: pos + batch_size], qwen_tokenizer)
        operations = refer_get_operation_for_icl(task_description, icl_sample_data)
        for v in operations:
            if len(v) > 2:
                task_operations.add(v)
# 持久化保存该特定问题的 operations
operations_list = [v for v in task_operations]
with open(operation_file, 'w') as f:
    json.dump(operations_list, f)
```

第三步：通过每个特定问题的 Operation 获得测试数据的预测结果 Result

FOR x IN test_examples:

FOR _ IN RANGE(layer_size):

X (输入) + Definition (问题定义或目标要求) + Operation



四、部署模型服务

系统：比赛官方提供的云服务器（为了加速实验 - 建议用更多的卡数，推荐用 8 个）

GPU：Ascend 910C x 2 卡

CPU：20 核

磁盘：200G

内存：80G

NPU：CANN8.3.RC2

查看环境 *cann* 信息：

```
cat /usr/local/Ascend/ascend-toolkit/latest/version.cfg
```

1、安装环境 (flagscale 所需 CANN、vllm、vllm-ascend 版本依赖 请按实际环境调整)

```
DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src
cd $DIR
pip install -r $DIR/requirements.txt
bash $DIR/create_env_ascend.sh
```

2、启动服务 (启动服务后需要耐心等待一小段时间, 让模型服务完全就绪, 可用 npu-smi 或 nputop 查看状态)

```
DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src
cd $DIR
bash $DIR/start-qwen3_4_server.sh
```

注: 默认配置单机多卡启动, 若超过 2 张 NPU, 请调整参数 tensor_parallel_size 数值

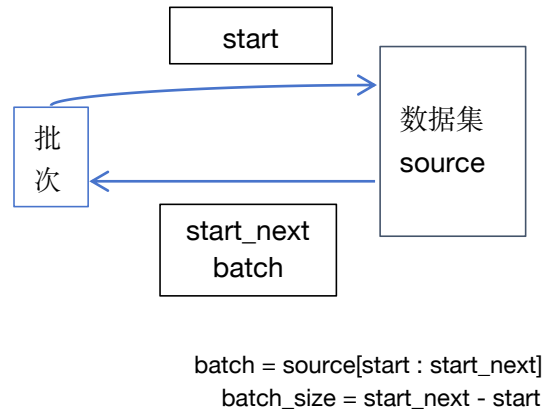
五、编码实现

1、模型上下文长度规划

1) 客户端提交 prompts 限制在 20000 左右

在拼接 ICL 示例数据或 Operation 数据时, 自动计算其 tokens 长度限制在 8192 以内, 辅助提示词限制 2048 以内, 提交 prompts 总长度限制在 10240 以内。并且将 ICL 示例数据自动划分为多个批次, 自动计算每个批次的最佳起止位置和示例个数, 让每个批次的 tokens 长度接近限制长度。伪代码如下:

```
strings = ""
tokens_len = 0
while pos <= stop:
    item = source[start]
    start += 1
    if tokens_len < 4096:
        substr = 拼接规则(item)
        tokens_len += 计算长度(substr)
        strings += substr
    else:
        break
return strings, start
```



2) 服务端生成输出限制在 10240 (8192+2048) 以内

在 API 提交请求时, 传参限制 max_tokens <= 10240(包含 think 内容和数据结果), 正文可以比 think 内容长, 但是 think 内容不能太长。若正文长度加 think 长度超过 10240, 就主动终止客户端请求连接截断输出, 以避免 think 异常的无限重复或无限增长。

3) 模型推理计算 tokens 总长度限制在 20480 以内, 部分参数如下:

gpu_memory_utilization: 0.4	预留安全余量以应对动态缓存增长
tensor_parallel_size: 1	单机 NPU 卡数, 推荐多卡并行
block_size: 128	升腾用 128 对于超长文推理更好
dtype: bfloat16	升腾用 bfloat16 才能保证稳定与精度, 并且更快, 绝不能用 float16
max_num_seqs: 1	服务并发请求处理个数 (也是同时计算批次个数)
max_model_len: 20480	安全的推理长度 (20480 >= 10240+10240)
max_num_batched_tokens: 20480	防止显存容量 OOM (max_num_seqs + 1) * max_model_len

注：为了尽量减少不同任务的 KV 缓存干扰其他任务推理结果，同时充分利用资源获得较快速度，所以设置最低配置参数，并且每张卡都启动 2 个服务。于是配置单机 2 卡 4 服务支持 4 个并行任务，从而尽量隔不同任务的离 KV 缓存：

```
llm_config_s4_0.yaml : port = 2026 ASCEND_RT_VISIBLE_DEVICES = 0
llm_config_s4_1.yaml : port = 2027 ASCEND_RT_VISIBLE_DEVICES = 0
llm_config_s4_2.yaml : port = 2028 ASCEND_RT_VISIBLE_DEVICES = 1
llm_config_s4_3.yaml : port = 2029 ASCEND_RT_VISIBLE_DEVICES = 1
```

若你的服务器环境卡很多，则可按需增大 max_num_seqs 和 tensor_parallel_size，用全部卡启动一个服务，全部全无串行执行。

2、利用 ICL 示例提取隐性的关键操作 Operation

启用 think 思考链模式，遍历 ICL 数据集，多次提交小批次的 ICL 示例数据生成 Operation，并且将各个批次生成的 Operation 合并去重，并且持久保存，icl_temperature = 0.7。

DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src

```
python $DIR/main_batch.py --task_id=0 --task_step=1
```

或者在后台运行

```
nohup python $DIR/main_batch.py --task_id=0 --task_step=1 > output.log 2>&1 &
```

查看进度 tail -10 \$DIR/icl_progress-{task_id}.log

结果保存 \$DIR/operations_tmp 目录，文件名称为：operation-{task_id}.json

3、利用 Operation 获取预测结果 Result

启用 think 思考链模式，加载对应的 Operation，遍历每个 Test 数据，并且与筛选的 Operation 一起提交给模型推理生成，test_temperature = 0.3。

DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src

```
python $DIR/main_batch.py --task_id=0 --task_step=2
```

或者在后台运行

```
nohup python $DIR/main_batch.py --task_id=0 --task_step=2 > output.log 2>&1 &
```

查看进度 tail -10 \$DIR/test_progress-{task_id}.log

结果保存 \$DIR/outputs 目录，文件名称为：openseek-{task_id}-v{version}.jsonl

4、检查 Result 异常数据

a) 解决非推理问题：可能是服务端未能正常返回，

b) 推理问题：可能是 KV 缓存因素，也可能是模型自身不稳定

这些问题，大都能通过重新请求解决，而且本程序已经设计实现了鉴别和处理这些异常的方法：

首先，在 API 返回解析时分析判断得到是否异常，并且会很快重复请求一次。

然后，在保存 Result 时，每个数据会记录一个 state 字段标记是否异常。例如

```
{"test_sample_id": "openseek-1-ed5ac69191204cd4bfb0ca41bc7f197f", "state": true, "reps": 0, "prediction": "8"}
```

最后，在对应任务的第二阶段执行完后，自动开始扫描结果文件，检查纠正异常数据，并且最多尝试 3 次。

5、执行命令说明

5.1) 命令参数

--task_id=0, 遍历处理所有任务

--task_id=\$id, 则 one of [1,2,3,4,5,6,7,8], 只处理对应的一个任务

--task_step=1, 则只是利用 ICL 样本提取隐性的关键操作 Operation

--task_step=2, 则只是利用 Operation 获取预测结果 Result

--task_step=0, 则首先利用 ICL 提取 Operation，然后利用 Operation 获取 Result

5.2) 入口文件

main.py 总是每次同步提交一个批次，便于前台调试查看过程进行观察分析，例如：

```
DEBUG_PRINT_STREAM=1 python main.py --task_id=5 --task_step=1
```

```
DEBUG_PRINT_STREAM=2 python main.py --task_id=5 --task_step=1
```

main_batch.py 可以每次异步并发提交多个批次，用于后台命令完成任务，加速实验进度，例如：

```
REQ_CONCURRENCY_NUM=8 python main_batch.py --task_id=5 --task_step=1
```

5.3) 环境变量，如果你采用了自定义参数配置，则需按实际修改默认值，否则默认即可

```
export SERVE_API_KEY=""
export SERVE_BASE_URL="http://localhost:2026/v1"
export SERVE_MODEL_NAME="Qwen/Qwen3-4B"
export SERVE_TOKENIZER_PATH=~/.cache/modelscope/hub/models/Qwen/Qwen3-4B"
```

5.4) 从头跑到尾的命令（单个服务执行这个命令）

```
DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src
```

```
bash $DIR/start_all_tasks_on_1_serve.sh
```

6、执行命令分析解读：需要考虑这 8 个任务数据 kv 缓存的相互干扰因素

在卡少时可以用 a 或 b 方式，本人`单机 2 卡部署成 4 个服务（每卡 2 个服务）`使用 c 方式。

本实验机器环境，采用的是 c 方式。考虑在并发时让不同任务 KV 缓存隔离，避免干扰

```
DIR=your-project-path/OpenSeek/openseek/competition/LongContext-ICL-Annotation/src
```

6.1 (a 方式) 1 个服务，逐个任务串行执行，因为 8 个任务的 kv 缓存相互干扰较小

```
cd $DIR && bash start-qwen3_1_serve.sh
```

```
cd $DIR && bash start-all-tasks_on_1_serve.sh
```

注：也可以在每处理一个后，重启服务清楚缓存，但是有点麻烦与耗时。

6.2 (b 方式) 2 个服务，两阶段分步执行（干扰比 a 少，速度比 a 快）

```
# 为了在并发时让不同任务 KV 缓存隔离，未使用 max_num_seqs>1 tensor_parallel_size>1
```

```
cd $DIR && bash start-qwen3_2_serve.sh
```

```
# 首先执行第一阶段
```

```
cd $DIR && bash start-all-tasks_on_2_serve-step_1.sh
```

```
# 然后启动第二阶段：自动循环监视，在发现第一步中出现第一个文件生成时，就自动开始第二阶段，在第一阶段完成后，自动接管其服务用于处理第二阶段的其他任务
```

```
cd $DIR && nohup bash start-all-tasks_on_2_serve-step_2.sh > s2-wait-s1.log 2>&1 &
```

6.3 (c 方式) 4 个服务，8 个任务 KV 缓存在不同服务，多任务并行执行，干扰很小速度较快

```
# 为了在并发时让不同任务 KV 缓存隔离，未使用 max_num_seqs>1 tensor_parallel_size>1
```

```
cd $DIR && bash start-qwen3_4_serve.sh
```

```
cd $DIR && bash start-all-tasks_on_4_serve.sh
```

6.4 (d 方式) 8 个服务，多个任务并行执行，8 个任务的 kv 缓存在 8 个不同服务互不干扰

```
# 仅作参考，最好时 8 张卡，每卡一个模型服务
```

```
cd $DIR && bash $DIR/start-all-tasks_on_8_serves.sh
```

六、未来优化方向

1、优化隐性的关键操作（Operation）的提取与应用

1) 优化对学习记忆 Operation 的存取

使用数据库和缓存技术存取 Operation，提供便捷的查阅和分析服务，便于用户理解与

分析。统计每次 **Operation** 的使用频次和输入要点关联，当提交文本长度受限时，优先使用频次高的或者让基于频次的选中概率更高的或者关联更高的 **Operation**。

2) 快速提取全新领域问题的 **Operation**

搭建一个服务平台，首先积累多种领域问题的 **Operation**，然后进行数据挖掘找到其中的生成模式，设计训练一个 **Operation** 模型，以达到在其他全新领域问题只需极少数据就能快速学习到领域问题的 **Operation**，提高项目的实用性。

2、探索关联关系(Association)的提取与应用

基于图数据库和联网查询能力，学习记忆关联关系，在新的输入信息推理时，将相关的关联关系一起提交给大模型分析，从而提高模型的知识整合能力和输出准确率，以适应现实世界的更多应用场景。

后面的附件只列出核心逻辑的核心代码，完整代码请前往查阅源码。

附件 1: 单机 2 卡 4 服，4 个配置 llm_config_s4_x.yaml

参数 port、exp_dir、ASCEND_RT_VISIBLE_DEVICES 不同

```
#llm_config_s4_0.yaml : port = 2026 exp_dir=s4_0 ASCEND_RT_VISIBLE_DEVICES = 0
#llm_config_s4_1.yaml : port = 2027 exp_dir=s4_1 ASCEND_RT_VISIBLE_DEVICES = 0
#llm_config_s4_2.yaml : port = 2028 exp_dir=s4_2 ASCEND_RT_VISIBLE_DEVICES = 1
#llm_config_s4_3.yaml : port = 2029 exp_dir=s4_3 ASCEND_RT_VISIBLE_DEVICES = 1
```

serve:

```
- serve_id: vllm_model
  engine: vllm
  engine_args:
    model: /root/.cache/modelscope/hub/models/Qwen/Qwen3-4B
    host: 0.0.0.0
    served-model-name: Qwen/Qwen3-4B
    uvicorn_log_level: warning
    port: 2026
    trust_remote_code: true
    gpu_memory_utilization: 0.4
    pipeline_parallel_size: 1
    tensor_parallel_size: 1
    block_size: 128
    max_num_seqs: 1
    max_model_len: 20480
    max_num_batched_tokens: 20480
    dtype: float16
    enforce_eager: true
    enable_prefix_caching: true
    enable_chunked_prefill: true
    disable_log_stats: true
    async_scheduling: true
    distributed_executor_backend: mp
```

experiment:

```
exp_name: qwen3_4b
exp_dir: s4_0/${experiment.exp_name}
task:
  type: serve
runner:
  hostfile: null
  deploy:
    use_fs_serve: false
envs:
  ASCEND_RT_VISIBLE_DEVICES: 0
  ASCEND_RT_DEVICE_MAX_CONNECTIONS: 2
```

action: run

hydra:

```
run:
  dir: ${experiment.exp_dir}/hydra
```

附件 2: 在 method.py 的部分核心逻辑代码

构建 Prompt:

利用 ICL 提取 Operation 的 Prompt

```
def make_icl_prompt(task_description: str, icl_sample_data: str):
```

```
    temperature = 0.7
```

```
    prompt = f"你是一名专业的数据标注专家，请仔细阅读理解示例数据的任务目标(在==左侧的<input>与</input>之间)与任务结果(在==右侧的<output>与</output>之间)，首先利用示例数据按照任务描述的要求对任务目标与任务结果进行推理和推导，并且要善于使用逆向思维、跳跃思维和横向关联以避免深陷在反复思考或递归循环之中，然后分析提取出从任务目标得到任务结果所必须执行的在 10 个以内的关键操作，并且必须是在对所有示例数据需要的普适通用的关键操作，并且必须与任务描述强相关的，并且必须与示例数据弱相关的，然后表示为言简意赅的纯英文提示词短语，不能包含引号、逗号、分号、句号、横线、表情等标点符号或特殊符号，不能包含数字或项目符号，不能是无关的或冗余的，不能是示例数据或解释内容，不能是假设或推测，并且必须用<operation>与</operation>包裹每个关键操作提示词，之后再一起填写在<output_answer>与</output_answer>之间。\\
```

```
    任务描述: {task_description} \\
```

```
    示例数据: {icl_sample_data} \\
```

```
    输出格式: <output_answer><operation></operation></output_answer>"
```

```
    return prompt, temperature
```

利用 Operation 和 Test 获取 Result 的 Prompt

```
def make_test_prompt(task_description: str, task_input: str, core_operations: str):
```

```
    temperature = 0.3
```

```
    prompt = f"你是一名永远都保持冷静与理性的解题高手，请仔细阅读理解任务描述、任务目标和关键操作(在每个<operation>与</operation>之间)，你必须一直保持冷静与理性地解决问题，不要让外界信息影响到你的情绪和判断，首先参考任务描述与关键操作进行联想和思考，从中找出最相关的关键操作，并且根据你的最佳经验对当前任务目标补充缺失的关键操作，然后按照任务描述的要求对任务目标进行推理和推导，并且要善于使用逆向思维、跳跃思维和横向关联以避免深陷在反复思考或递归循环之中，并且按照任务描述的规范生成任务目标的任务结果，并且其格式与数据类型必须严格符合任务描述的规范与要求，结果不能是假设或推测，结果不能是重复罗嗦或敷衍，结果不能是简单直白或口述，如果任务描述没有要求包含步骤或解释那么结果就不能包含步骤或解释，最后必须将任务结果全用英文表达作为标准答案，并且填写在<output_answer>与</output_answer>之间。\\
```

```
    任务描述: {task_description} \\
```

```
    任务目标: {task_input} \\
```

```
    隐性的关键操作: {core_operations} \\
```

```
    输出格式: <output_answer></output_answer>"
```

```
    return prompt, temperature
```

对 ICL 示例数据的长文拼接逻辑:

批量拼接 ICL 示例数据 (implementation of Long-Context Data Annotation)

```
def select_examples(icl_examples: list[dict], tokenizer: AutoTokenizer, start: int, stop: int, target_length=8192) -> str:
```

```
    """ Select examples from icl_examples to fit into the target context length (适配 Qwen3-4B 的 token 计算).
```

```
    icl_examples:
```

```
        A list of examples, where each example is a dict with keys 'input' and 'output' (no 'length' needed).
```

```
        For example, "{ \"input\": \"The material is good and looks great.\", \"output\": \"Good Review\" }",
```

```
    tokenizer:
```

```
        Qwen3-4B 的分词器
```

```
    start:
```

```
        最小起始位置,范围[0,len(icl_examples)-1]
```

```
    stop:
```

```
        最大结束位置,范围[0,len(icl_examples)-1]
```

```
    target_length:
```

```
        上下文长度限制 (Qwen3-4B 的上下文窗口默认是 8k/32k, 根据实际调整, 若比较严格则建议为 8192)
```

```
    """
```

```
    total = len(icl_examples)
```

```
    assert total > 0 and start >= 0 and stop >= start and start < total and stop < total, f"must be `total > 0 and start >= 0 and stop >= start and start < total and stop < total`, error: {start}, {stop}, {total}"
```

```
    examples_str, token_num = "", 0
```

遍历所有示例，基于 Qwen3-4B 的 tokenizer 计算 token 数

```
while start <= stop:
```

```
    example = icl_examples[start]
```

```
    start += 1
```

提取 input 和 output(兼容 output 是列表的情况)

```
input_text = example['input']
```

```
output_text = example['output'][0]
```

核心: 用 Qwen3-4B 的 tokenizer 计算 input+output 的 token 数(替代原 length 键)

encode 返回 token id 列表, len 即为 token 数

```
input_tokens = len(tokenizer.encode(input_text, add_special_tokens=False))
```

```
output_tokens = len(tokenizer.encode(output_text, add_special_tokens=False))
```

```
length = input_tokens + output_tokens # 等效原示例的 length 值
```

```

# 校验当前示例是否能加入(总长度不超限制)
if length + token_num <= target_length:
    # 累加总 token 数: 示例文本长度 + 格式符号的 token 数(<input>7 + </input>==<output>18 + </output>9)
    token_num += (length + 7 + 18 + 9)
    # 拼接示例字符串
    examples_str += f"<input>{input_text}</input>==<output>{output_text}</output>"
else:
    # 超过长度限制, 当前截止拼接
    break

# 返回已拼接的示例和已选数量, 作为当前批次
return examples_str, start

```

对 Operations 提示词的长文拼接逻辑:

```

# 批量拼接隐性的关键操作提示词 operations
def select_operations(task_operations: list[dict], tokenizer: AutoTokenizer, start: int, stop: int, target_length=8192) -> str:
    """
    Select operations from task_operations to fit into the target context length (适配 Qwen3-4B 的 token 计算).
    task_operations:
        A list of operations, where each operation is a string.
    tokenizer:
        Qwen3-4B 的分词器
    start:
        最小起始位置,范围[0,len(task_operations)-1]
    stop:
        最大结束位置,范围[0,len(task_operations)-1]
    target_length:
        上下文长度限制 (Qwen3-4B 的上下文窗口默认是 8k/32k, 根据实际调整, 若比较严格则建议为 8192)
    """
    total = len(task_operations)
    assert total > 0 and start >= 0 and stop >= start and start < total and stop < total, f"must be `total > 0 and start >= 0 and stop >= start and start < total and stop < total`, error: {start}, {stop}, {total}"
    operations_str, token_num = "", 0
    # 遍历所有示例, 基于 Qwen3-4B 的 tokenizer 计算 token 数
    while start <= stop:
        operation = task_operations[start]
        start += 1
        # 核心: 用 Qwen3-4B 的 tokenizer 计算 input+output 的 token 数(替代原 length 键)
        # encode 返回 token id 列表, len 即为 token 数
        length = len(tokenizer.encode(operation, add_special_tokens=False))
        # 校验当前示例是否能加入(总长度不超限制)
        if length + token_num <= target_length:
            # 累加总 token 数: 示例文本长度 + 格式符号的 token 数(<operation>11 + </operation>12)
            token_num += (length + 11 + 12)
            # 拼接单个示例字符串
            operations_str += f"<operation>{operation}</operation>"
        else:
            # 超过长度限制, 当前截止拼接
            break
    # 返回已拼接的示例和已选数量, 作为当前批次
    return operations_str, start

```

对模型返回内容的解析逻辑与规则:

```

# 获取隐性的关键操作的提示词
def parse_operations(answer: str) -> list:
    """
    提取字符串中<operation>标签内的所有内容(字符串形式), 统计出现次数最多的内容
    :answer: 包含<operation>标签的原始字符串
    :return: 去掉标签或分隔符之后的文本数组
    """
    if answer == None:
        return None, False
    operations = _parse_operations_from_str(answer)
    if len(operations) == 0:
        return None, False
    content_counter = Counter(operations)
    if not content_counter:
        return None, False
    # 提取最高频次的前 10 个
    max_count = max(content_counter.values())
    sel_count = max_count - 10
    operations = [content.strip() for content, count in content_counter.items() if count > sel_count]
    return operations, True

# 获取测试数据的预测结果
def parse_result(answer: str, return_data_type: str):
    if answer == None:
        return None, False

    match return_data_type:
        # 数字
        case 'number':

```

```

        arr = __parse_number(answer)
    # 包含多种符号的任意英文数组
    case 'array':
        arr = __parse_array(answer)
    # 包含多种符号的任意英文字符串
    case 'text':
        return answer, True
    case _:
        return answer, True
data_counter = Counter(arr)
if not data_counter:
    return None, False
# 提取最高频次的数值作为最终答案
max_count = max(data_counter.values())
result = [ data.strip() for data, count in data_counter.items() if count == max_count ]
return result[0], True

# 提取隐性的关键操作的英文短语
def __parse_operations_from_str(text: str):
    if text.find('<operation>') > -1:
        arr = []
        # 首个是空字符或者无关信息
        for s in text.split('<operation>')[1 : None]:
            s = s.split('</operation>')[0].strip()
            # 标点符号通常用作分隔符 ('-\s/ 往往也是操作短语的组成部分)
            arr.extend(re.split(r'[.,:;!@#%&*\()\[\]<>~^"\\s\n\t]', s))
    else:
        # 标点符号通常用作分隔符 ('-\s/ 往往也是操作短语的组成部分)
        arr = re.split(r'[.,:;!@#%&*\()\[\]<>~^"\\s\n\t]', text)
    return [ s.strip() for s in arr if len(s) > 3 and not re.search(r'\d(inappropriate)', s) ]

# 提取英文数值,例如: -0.9, 12.33, 10_000
def __parse_number(text: str) -> list:
    arr = []
    # . 是小数点 - 是负号 _ 也是英文数字连接
    for s in re.split(r'[.,:;!@#%&*\()\[\]<>~^"\\s\n\t]', text):
        arr.extend(re.findall(r'[0-9-]+[0-9._]*', s.strip()))
    return arr

# 提取英文数组
def __parse_array(text: str) -> list:
    return re.findall(r'\[[a-zA-Z0-9.,:;!@#%&*\()\[\]<>~^"\\s\n\t]+\]', text.strip())

```

附件 3: 在 api_client.py 的部分核心逻辑代码, 请求模型服务 API

```

serve_api_key = os.environ.get('SERVE_API_KEY', '')
serve_base_url = os.environ.get('SERVE_BASE_URL', 'http://localhost:2026/v1')
serve_model_name = os.environ.get('SERVE_MODEL_NAME', 'Qwen/Qwen3-4B')
debug_print_stream = os.environ.get('DEBUG_PRINT_STREAM', 0)

# 多个并发请求模型服务 (流式)
def batch_request_llm_api_stream(prompts: list, max_tokens=10240, temperature=0.6, enable_thinking=True):
    answers = []
    texts = asyncio.run(ChatClient.__batch_request_chat_api_stream(prompts, max_tokens, temperature, enable_thinking))
    for i, (text, state) in enumerate(texts):
        if debug_print_stream == str(CONST_DEBUG_SHOW_ASNSWER) or debug_print_stream ==
CONST_DEBUG_SHOW_ASNSWER:
            print('text, state = ', text, state)
        # 若格式不符合要求, 则多请求几次尽量提高成功率
        if state == False or text.find('<output_answer>') == -1:
            time.sleep(3)
            text, state = asyncio.run(ChatClient.__request_chat_api_stream(prompt, max_tokens, temperature,
enable_thinking, 512))
        if debug_print_stream == str(CONST_DEBUG_SHOW_ASNSWER) or debug_print_stream ==
CONST_DEBUG_SHOW_ASNSWER:
            print('text, state = ', text, state)
        if state == False:
            text, _ = ChatClient.__parse_answer(text)
            answers.append(text, state)
        else:
            answers.append(ChatClient.__parse_answer(text))
    return answers

# 单个请求模型服务 (流式)
def request_llm_api_stream(prompt: str, max_tokens=10240, temperature=0.7, enable_thinking=True):
    text, state = asyncio.run(ChatClient.__request_chat_api_stream(prompt, max_tokens, temperature, enable_thinking))
    if debug_print_stream == str(CONST_DEBUG_SHOW_ASNSWER) or debug_print_stream ==

```

```

CONST_DEBUG_SHOW_ASNSWER:
    print('request_llm_api_stream-p1: text, state = ', text, state)
    # 若格式不符合要求, 则多请求一次尽量提高成功率
    if state == False or text.find('<output_answer>') == -1:
        time.sleep(3)
        text, state = asyncio.run(ChatClient.__request_chat_api_stream(prompt, max_tokens, temperature, enable_thinking,
512))
        if debug_print_stream == str(CONST_DEBUG_SHOW_ASNSWER) or debug_print_stream ==
CONST_DEBUG_SHOW_ASNSWER:
            print(' text, state = ', text, state)
    if state == False:
        text, _ = ChatClient.__parse_answer(text)
        return text, state
    else:
        return ChatClient.__parse_answer(text)

# 获取任务结果英文文本 (可以包含分隔符和特殊标签)
def __parse_answer(text: str) -> str:
    """
    提取字符串中<output_answer>标签内的所有内容(字符串形式), 统计出现次数最多的内容
    :text: 包含<output_answer>标签的原始字符串
    :return: 最后一个完整<output_answer></output_answer>之内的数据
    """
    if text == None:
        return None, False
    if text.find('<output_answer>') > -1:
        output_answer = text.split('<output_answer>')[-1].split('</output_answer>')[0].strip()
    else:
        # 返回格式可能不符合预期, 则用正则提取英文内容,
        # 在 Prompts 已经要求返回英文结果(可包含常见的特殊英文字符, 支持返回编程代码)
        pre_answers = re.findall(r"[a-zA-Z0-9,.;:!?@#%&*\[\]\{\}<>|/~'\^\"\\-\s\n\t]+", text)
        if len(pre_answers) == 0:
            return None, False
        # 排除空白
        answers = []
        for a in pre_answers:
            a = a.strip()
            if a != '':
                answers.append(a)
        if len(answers) == 0:
            return None, False
        # 但此时只要提取最后一个英文内容 (答案通常都是输出在最后)
        output_answer = answers[-1]
    return output_answer, True

# 并发请求模型服务 (流式)
async def __batch_request_chat_api_stream(prompts: list, max_tokens=10240, temperature=0.6, enable_thinking=True):
    tasks = []
    for prompt in prompts:
        tasks.append(ChatClient.__request_chat_api_stream(prompt, max_tokens, temperature, enable_thinking))
    return await asyncio.gather(*tasks)

# 模型服务流式输出
async def __request_chat_api_stream(prompt: str, max_tokens=10240, temperature=0.6, enable_thinking=True,
len_for_exception_response=0):
    client = AsyncOpenAI(
        api_key = serve_api_key,
        base_url = serve_base_url.rstrip('/'),
        timeout = 60*60*24,
    )
    messages = [
        {
            "role": "system",
            "content": "你是一名无所不知无所不能的全能高手, 请仔细理解任务描述和任务目标, 展开所有联想找到最佳策略和方法, 尽你所能生成用户期望的标准答案, 返回结果的格式也须符合用户要求。"
        },
        {
            "role": "user",
            "content": str(prompt) if enable_thinking else str(prompt) + '/no_think', # 必须强制转一次字符串
        }
    ]

```

```

    }
]
try:
    response = await client.chat.completions.create(
        model = serve_model_name,
        messages = messages,
        temperature = temperature,
        max_tokens = max_tokens,
        stream = True,
    )

    # 粗略估计字符串长度限制
    max_len = 3 * max_tokens
    full_len = 0
    full_content = ""
    async with response:
        async for chunk in response:
            content = None
            if hasattr(chunk.choices[0].delta, 'reasoning_content') and chunk.choices[0].delta.reasoning_content:
                content = chunk.choices[0].delta.reasoning_content
            elif hasattr(chunk.choices[0].delta, 'content') and chunk.choices[0].delta.content:
                content = chunk.choices[0].delta.content
            else:
                pass

            if content != None:
                full_content += content
                # 利用 env:DEBUG_PRINT_STREAM 判断是否实时打印 (调试查看模型的输出内容)
                if debug_print_stream == str(CONST_DEBUG_SHOW_STREAM) or int(debug_print_stream) ==
CONST_DEBUG_SHOW_STREAM:
                    print(content, end="", flush=True)

                # 若发现长度异常, 则主动终止连接提前截断输出, 在这里粗略估计即可
                full_len += len(content)
                if full_len > max_len:
                    await client.close()
                    break

    text = full_content.strip()

    # 处理异常数据: 长度截断、者胡言乱语、无限重复等等
    exists_think_tag = text.find('</think>') > -1
    exists_output_answer_tag = text.find('<output_answer>') > -1
    if not exists_think_tag and not exists_output_answer_tag:
        if len_for_exception_response <= 0:
            return None, False
        else:
            return text[- len_for_exception_response : None ], False
    else:
        # 去掉 think 内容
        if exists_think_tag:
            text = text.split('</think>', 1)[1].strip()
        # "</output_answer>"可能没有返回, 则自动补全
        if exists_output_answer_tag and text[-16 : None] != "</output_answer>":
            text += "</output_answer>"
        return text, True

except APIConnectionError as e:
    print(f"连接失败: {e}")
    print("请检查 base_url 是否配置正确, 或者网络是否正常。")
    raise e
except AuthenticationError as e:
    print(f"鉴权失败: {e}")
    print("请检查 API Key 是否填写正确。")
    raise e
except APIError as e:
    error = str(e)
    if error.find("inappropriate") > 0:
        return '<output_answer>Output data may contain inappropriate content</output_answer>'

```

```

        else:
            raise e
    except BadRequestError as e:
        # Error code: 400 - Input data may contain inappropriate content.
        error = str(e)
        if error.find("inappropriate") > 0:
            return '<output_answer>Input data may contain inappropriate content</output_answer>'
        else:
            raise e
    except Exception as e:
        error = str(e)
        if error.find("inappropriate") > 0:
            return '<output_answer>Output data may contain inappropriate content</output_answer>'
        else:
            raise e

```

附件 4: 在 main.py 与 main_batch.py 的部分核心逻辑代码

在 main.py 单个请求提交 ICL 示例的核心代码:

```

# 遍历所有示例样本，自动计算合适长度的批次
start = start if start >= 0 else 0
stop = stop if stop >= start else len(icl_examples) - 1
while start <= stop:
    icl_batch_data, start_next = select_examples(icl_examples, qwen_tokenizer, start, stop,
CONST_LIMIT_OUTPUT_MAX_TOKENS)

    print("\n")
    print_icl_progress(task_id, 'icl_batch_data-strlen', len(icl_batch_data), "start", start, "batch_size", start_next -
start, "task", task_id, task_name)
    start = start_next

    operations, state = refer_get_operation_for_icl(task_description, icl_batch_data,
CONST_LIMIT_OUTPUT_MAX_TOKENS)

    print("\n-----")
    if operations == None:
        print_icl_progress(task_id, f"The batch get operations is None, state: {state}", )
    else:
        print_icl_progress(task_id, f"The batch get operations is Success, state: {state}", operations)
        for v in operations:
            task_operations.add(v)

```

在 main.py 单个请求提交 TEST 的核心代码:

```

# 无论有多少隐性的关键操作，都只提交推理一次测试，所以也只遍历一次隐性的关键操作，简单获取足量的关键操作即可
random.shuffle(task_operations)
core_operations, _ = select_operations(task_operations, qwen_tokenizer, 0, len(task_operations) - 1,
CONST_LIMIT_PROMPT_MAX_TOKENS)

start = start if start >= 0 else 0
stop = stop if stop >= start else len(test_samples) - 1
for i in range(start, stop + 1):
    print(f'Evaluation item={i} on Task {task_id}: {task_name}')

    test_sample_id = test_samples[i]['id']
    test_input = test_samples[i]['input']
    prediction, state = refer_get_result_for_test(task_description, test_input, core_operations,
CONST_TASK_TEST_RETURN_DATA_TYPES[task_id], CONST_TASK_OUTPUT_MAX_TOKENS_SIZES[task_id])

    print("\n-----")
    print_test_progress(task_id, f"Answer-item={i}:", prediction)

# 保存测试结果
test_record = {'test_sample_id': test_sample_id, "state": state, 'reps': 0, 'prediction': prediction}
with open(result_file, 'a') as f:
    f.write(json.dumps(test_record)+'\n')

```

在 main_batch.py 批量并发请求提交 ICL 示例的核心代码:

```
for i in range(N):
    # 省略这之间其他代码...
    # 遍历所有示例样本, 自动计算合适长度的批次
    icl_batch_data_list = []
    start = start if start >= 0 else 0
    stop = stop if stop >= start else len(icl_examples) - 1
    while start <= stop:
        if len(icl_batch_data_list) < req_concurrency_size and start < stop:
            icl_batch_data, start_next = select_examples(icl_examples, qwen_tokenizer, pos,
CONST_LIMIT_PROMPT_MAX_TOKENS)
            log_icl_progress(task_id, 'icl_batch_data-strlen', len(icl_batch_data), "start", start, "icl_batch_size",
start_next - start, "task", task_id, task_name)
            start = start_next
            icl_batch_data_list.append(icl_batch_data)
        else:
            operations_list = batch_refer_get_operation_for_icl(task_description, icl_batch_data_list,
CONST_LIMIT_OUTPUT_MAX_TOKENS)
            icl_batch_data_list = []

            for operations, state in operations_list:
                if operations == None:
                    log_icl_progress(task_id, f"The batch {i}::{j}-req_concurrency_size-{j} get operations is None,
state: {state}")
                else:
                    log_icl_progress(task_id, f"The batch {i}::{j}-req_concurrency_size-{j} get operations is
Success, state: {state}", operations)
                    for v in operations:
                        task_operations.add(v)

            # 写入持久化文件
            operations_list = [v for v in task_operations]
            log_icl_progress(task_id, 'operations_list-size', len(operations_list))

            if len(operations_list) > 0:
                with open(operation_file, 'w') as f:
                    json.dump(operations_list, f)
```

在 main_batch.py 批量并发请求提交 TEST 的核心代码:

无论有多少隐性的关键操作, 都只提交推理一次测试, 所以也只遍历一次隐性的关键操作, 简单获取足量的关键操作即可

```
random.shuffle(task_operations)
core_operations, _ = select_operations(task_operations, qwen_tokenizer, 0, len(task_operations) - 1,
CONST_LIMIT_PROMPT_MAX_TOKENS)
# 省略这之间其他代码...
# 提交测试
start = start if start >= 0 else 0
stop = stop if stop >= start else len(test_samples) - 1
for i in range(start, stop + 1, req_concurrency_size):
    log_test_progress(task_id, f'Evaluation items({i}::{i+req_concurrency_size}) on Task {task_id}: {task_name}')
    test_sample_id_list = []
    test_input_list = []
    for test_sample in test_samples[i: i + req_concurrency_size]:
        test_sample_id_list.append(test_sample['id'])
        test_input_list.append(test_sample['input'])
    prediction_list = batch_refer_get_result_for_test(task_description, test_input_list, core_operations,
CONST_TASK_TEST_RETURN_DATA_TYPES[task_id], CONST_TASK_OUTPUT_MAX_TOKENS_SIZES[task_id])

    j = 0
    for test_sample_id, (prediction, state) in zip(test_sample_id_list, prediction_list):
        log_test_progress(task_id, f"Answer-item={i+j}:", prediction)
        j += 1
    # 保存测试结果
    test_record = {'test_sample_id': test_sample_id, 'state': state, 'reps': 0, 'prediction': prediction}
    with open(result_file, 'a') as f:
        f.write(json.dumps(test_record)+'\n')
```

在 main_batch.py 检查 Result 异常数据的核心代码:

```
# 自动检查异常数据: 可能是服务端为正常返回, 也可能是 KV 缓存, 也可能是模型不稳定
# 像这样的异常数据, 可用通过重新请求服务, 最终得到正常输出
def check_evaluate_correct(task_id:int, qwen_tokenizer:AutoTokenizer, reps = 0):
    assert task_id in [i for i in range(1, 9)], f"task_id should be in [1, 8], but got {task_id}."
    # 测试结果保存位置
    version = 1
    result_file = get_result_filepath(task_id, version)
    result_dir = os.path.dirname(result_file)
    if not os.path.exists(result_dir):
        # 没有结果文件, 无需检查
        print(f'task_id={task_id}暂无对厅结果文件, 无需检查')
        return
    min_len, max_len = CONST_TASK_CHECK_STRLLEN_SIZES[task_id]['min'],
    CONST_TASK_CHECK_STRLLEN_SIZES[task_id]['max']
    checks = {}
    outputs = []
    with open(result_file, 'r') as file:
        # 逐行读取并解析 JSON
        i = 0
        for line in file:
            # 解析每行的 JSON 数据
            data = json.loads(line)
            outputs.append(data)
            # 检查是否满足特定条件
            data_len = len(data['prediction']) if data['prediction'] != None else 0
            if data['state'] == False or data_len < min_len or data_len > max_len:
                checks[data['test_sample_id']] = i
            # next
            i += 1
    if len(checks.values()) == 0:
        # 没有异常数据, 无需检查
        print(f'task_id={task_id}没有异常数据, 无需检查')
        return
    # 加载 Operation
    operation_file = get_operation_filepath(task_id)
    log_test_progress(task_id, 'operation_file', operation_file)
    with open(operation_file, 'r') as f:
        task_operations = json.load(f)
    # 无论有多少隐性的关键操作, 都只提交推理一次测试, 所以也只遍历一次隐性的关键操作, 简单获取足量的关键操作即可
    random.shuffle(task_operations)
    core_operations, _ = select_operations(task_operations, qwen_tokenizer,
    CONST_LIMIT_PROMPT_MAX_TOKENS)
    log_test_progress(task_id, 'core_operations-strlen', len(core_operations))
    # 加载任务文件
    task_file = get_task_filepath(task_id)
    with open(task_file, 'r') as f:
        task_dict = json.load(f)
    task_name = task_dict['task_name']
    task_description = task_dict['Definition'][0]
    test_samples = task_dict['test_samples']
    log_test_progress(task_id, 'test_samples-size', len(test_samples), "task", task_id, task_name)
    check_ids = checks.keys()
    indexes = [i for i, item in enumerate(test_samples) if item['id'] in check_ids]
    for k in range(0, len(indexes), req_concurrency_size):
        sample_idxes = indexes[k : k + req_concurrency_size]
        test_sample_id_list = []
        test_input_list = []
        for i in sample_idxes:
            test_sample_id_list.append(test_samples[i]['id'])
            test_input_list.append(test_samples[i]['input'])
        prediction_list = batch_refer_get_result_for_test(task_description, test_input_list, core_operations,
    CONST_TASK_TEST_RETURN_DATA_TYPES[task_id], CONST_TASK_OUTPUT_MAX_TOKENS_SIZES[task_id])
        j = 0
        for test_sample_id, (prediction, state) in zip(test_sample_id_list, prediction_list):
            log_test_progress(task_id, f"Correct answer-{sample_idxes[j]}:", prediction)
            j += 1
```

```

        if len(prediction) > max_len:
            prediction = prediction[ - (max_len + 5) : None ]
        # 保存测试结果
        outputs[checks[test_sample_id]] = {'test_sample_id': test_sample_id, 'state': state, 'reps': reps,
'prediction': prediction}
    new_result_file = get_result_filepath(task_id, version + 1)
    with open(new_result_file, 'w') as f:
        f.write('')
    with open(new_result_file, 'a') as f:
        for data in outputs:
            f.write(json.dumps(data)+'\n')
    # 覆盖旧文件
    os.rename(new_result_file, result_file)

```

附件 5: 在 const.py 定义的常量

```

# 限制输出长度
CONST_LIMIT_PROMPT_MAX_TOKENS = 8192
# 限制输出长度
CONST_LIMIT_OUTPUT_MAX_TOKENS = 10240
# 调试实时打印流式输出
CONST_DEBUG_SHOW_STREAM = 1
# 调试打印正文结果 (排除 think 后的 answer)
CONST_DEBUG_SHOW_ANSWER = 2
# 任务数据文件名称
CONST_TASK_FILES = {
    1: 'openseek-1_closest_integers.json',
    2: 'openseek-2_count_nouns_verbs.json',
    3: 'openseek-3_collatz_conjecture.json',
    4: 'openseek-4_conala_concat_strings.json',
    5: 'openseek-5_semeval_2018_task1_tweet_sadness_detection.json',
    6: 'openseek-6_mnli_same_genre_classification.json',
    7: 'openseek-7_jeopardy_answer_generation_all.json',
    8: 'openseek-8_kernel_generation.json',
}
# 模型输出 tokens 长度限制
CONST_TASK_OUTPUT_MAX_TOKENS_SIZES = {
    1: 8192 + 8,
    2: 8192 + 8,
    3: 8192 + 128,
    4: 8192 + 128,
    5: 8192 + 16,
    6: 8192 + 16,
    7: 8192 + 128,
    8: 8192 + 2048,
}
# 输出答案的字符串长度范围 (注: 既不包含 think 长度, 也不是 tokens 长度)
CONST_TASK_CHECK_STRLEN_SIZES = {
    1: { 'min': 1, 'max': 8 },
    2: { 'min': 1, 'max': 8 },
    3: { 'min': 1, 'max': 256 },
    4: { 'min': 1, 'max': 256 },
    5: { 'min': 1, 'max': 32 },
    6: { 'min': 1, 'max': 4 },
    7: { 'min': 1, 'max': 256 },
    8: { 'min': 128, 'max': 4096 },
}
# TEST 预测返回类型
CONST_TASK_TEST_RETURN_DATA_TYPES = {
    1: 'number',      # 例如: 12, 3, 5, -1, -0.36 等数字
    2: 'number',
    3: 'array',       # 例如: [1,2,3], ['a','b', 'c'] 等数组
    4: 'text',        # 任意英文字符串
    5: 'text',
    6: 'text',
    7: 'text',
    8: 'text',
}

```